Image recognition, a core application of deep learning, is typically performed using **Convolutional Neural Networks (CNNs)**.
Here is a complete, runnable Python example using **TensorFlow/Keras** to build, train, and evaluate a simple CNN for recognizing handwritten digits from the **MNIST** dataset.

Python Example: Image Recognition with CNN (MNIST)

This example covers all the necessary steps: data loading, preprocessing, model definition, training, and evaluation.

Prerequisites

You need to have **TensorFlow** and **NumPy** installed:

Bash

```
pip install tensorflow numpy matplotlib
```

Complete Code

Python

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Load and Preprocess Data ---

# Load the MNIST dataset (handwritten digits 0-9)
# The data is split into 60,000 training images and 10,000 testing images
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# 1.1 Reshape Data for CNN Input
# CNNs expect a 4D input: (samples, height, width, channels)
# MNIST images are 28x28 grayscale, so channels = 1
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32')
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32')

# 1.2 Normalize Pixel Values
# Convert pixel values from 0-255 to 0-1 for better model performance
x_train /= 255.0
x_test /= 255.0

# 1.3 One-Hot Encode Labels
# Convert integer labels (0-9) to categorical vectors (e.g., 5 -> [0,0,0,0,0,1,0,0,0,0])
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)

print(f"Training data shape: {x_train.shape}")
print(f"Testing data shape: {x_test.shape}")
print(f"Number of classes: {num_classes}")

# --- 2. Define the CNN Model Architecture ---

def build_cnn_model():
    model = Sequential([
        # Convolutional Layer 1: Learns features like edges, curves
        Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
        # Pooling Layer 1: Reduces spatial dimensions, making the model more robust
        MaxPooling2D((2, 2)),

        # Convolutional Layer 2
        Conv2D(64, (3, 3), activation='relu'),
        # Pooling Layer 2
        MaxPooling2D((2, 2)),

        # Flatten the 2D feature maps into a 1D vector for the Dense layers
        Flatten(),

        # Fully Connected (Dense) Layer 1
        Dense(128, activation='relu'),
        # Dropout: Randomly sets inputs to 0 to prevent overfitting
```

```
        Dropout(0.5),

        # Output Layer: 10 units for 10 classes, softmax for probability distribution
        Dense(num_classes, activation='softmax')
    ])

    # Compile the model
    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy', # Loss function for multi-class classification
        metrics=['accuracy']
    )
    return model

# --- 3. Train the Model ---

cnn_model = build_cnn_model()
print("\n--- Model Summary ---")
cnn_model.summary()

print("\n--- Starting Training ---")
history = cnn_model.fit(
    x_train, y_train,
    epochs=5,               # Number of passes over the entire dataset
    batch_size=128,
    validation_split=0.1    # Use 10% of training data for validation
)

# --- 4. Evaluate the Model ---

print("\n--- Evaluating Model on Test Data ---")
loss, accuracy = cnn_model.evaluate(x_test, y_test, verbose=0)

print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")

# --- 5. Predict on a New Image (Example) ---

# Take the first image from the test set
sample_image = x_test[0]
true_label_index = np.argmax(y_test[0]) # Get the true class index

# Keras expects a batch of images, even if it's just one, so add a batch dimension
```

```
sample_image_batch = np.expand_dims(sample_image, axis=0)

# Make the prediction
predictions = cnn_model.predict(sample_image_batch)
predicted_label_index = np.argmax(predictions[0]) # Get the index of the highest probability

print(f"\nSample Image True Label: {true_label_index}")
print(f"Sample Image Predicted Label: {predicted_label_index}")

# Optional: Plot the sample image
plt.imshow(sample_image.reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {predicted_label_index}, True: {true_label_index}")
plt.axis('off')
plt.show()
```